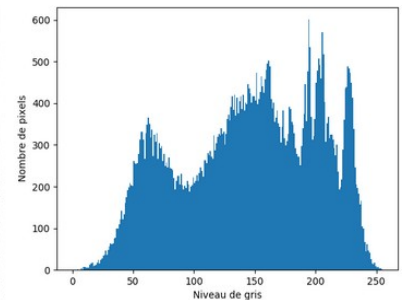


Filtrage d'image

Le principe du filtrage est de modifier la valeur des pixels d'une image, généralement dans le but d'améliorer son aspect ou d'en changer l'esthétique. Chaque pixel transformé est calculé à partir du pixel d'origine.

Il existe aussi du filtrage global comme les opérations sur les histogrammes. Ce type de filtrage ne sera pas traité ici.

Pour information : l'histogramme d'une image numérique est une courbe statistique représentant la répartition de ses pixels selon leur intensité. Pour une image en noir et blanc, il indique en abscisse le niveau de gris (entier entre 0 et 255) et en ordonnée, le nombre de pixels ayant cette valeur.



1. But de l'activité

→ Développer une classe de filtres d'images

2. Classe de manipulation d'image

Lors de cette activité nous manipulerons des images matricielles, c'est-à-dire représentées par des tableaux de pixels.

Nous utiliserons la bibliothèque Pillow pour s'affranchir de la question des formats de fichiers. Les principales commandes pour manipuler un fichier images sont les suivantes :

Affectation et affichage d'un fichiers d'image avec Pillow

Le tableau de valeurs des pixels de l'image est affecté à la variable `img`. Si l'on veut afficher l'image, on utilise la méthode `show()`

```
1 import PIL.Image as Image
2 img = Image.open('tigre.jpg')
3 img.show()
```

Informations sur une image

Dans un interpréteur Python, saisir et exécuter le code suivant :

```
1 import PIL.Image as Image
2 img = Image.open('tigre.jpg')
```

Puis utiliser la documentation interne de python (par exemple via la fonction `dir` de Python ou la commande `help` de l'interpréteur) pour explorer les informations fournies par l'objet image. Essayer par exemple d'obtenir la taille de l'image.



Représentation d'une image en mémoire

Si `img` est une image chargée avec `PIL.Image.open`, on accède à ses pixels via la méthode `img.load()` qui renvoie un tableau indexé par des couples d'entiers (et non pas une matrice au sens python du terme).

Par exemple :

```
pixels = img.load()
print(pixels[0,0])
```

Modifier une image

Pour modifier un pixel, on change sa valeur dans le tableau des pixels :

```
pixels = img.load()
pixels[0, 0] = (0, 0, 0)
```

La modification de la couleur d'un pixel dans l'objet image `img` s'effectue par la commande :

```
img.putpixel((x, y), (r, v, b))
```

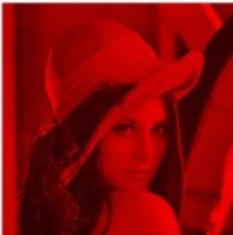
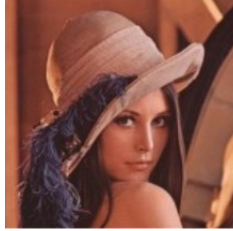
3. Classe filtre

On se propose de développer la classe `Filtre` décrite par le diagramme de classe suivant :

Filtre
- <code>img</code> : Image - <code>pix</code> : list
+ <code>__init__(img : Image)</code> + <code>size()</code> : tuple + <code>width()</code> : int + <code>height()</code> : int + <code>weight()</code> : int + <code>get_pix(col, : int, row : int)</code> : tuple + <code>reverse()</code> + <code>red()</code> + <code>color2grey()</code> + <code>threshold(limit : int)</code> + <code>show_image()</code> - <code>convert_pixel2img()</code>

Cette classe met en œuvre différents traitements dont les caractéristiques sont présentées en page suivante.



Nom de la méthode	Type de filtrage	Description	Filtrage effectué
<code>reverse()</code>	Négatif	Remplace tous les pixels de l'image par leur valeur en « négatif ». Obtenir le négatif d'une image est très simple : toutes les composantes x de tous les pixels de l'image sont remplacées par $255 - x$.	 
<code>red()</code>	Rouge	Chaque pixel de l'image est une combinaison de rouge, de vert et de bleu. En assignant la valeur 0 aux composantes verte et bleue, on obtient une image à dominante rouge.	 
<code>color2grey()</code>	Niveau de gris	L'œil est plus sensible à certaines couleurs qu'à d'autres et notamment au vert. Pour tenir compte de cette sensibilité dans la transformation d'une image couleur en une image en niveaux de gris, on ne prend pas la moyenne arithmétique des canaux rouge, vert et bleu, mais une moyenne pondérée. Pour simplifier, nous prendrons ici la moyenne « classique ».	 
<code>threshold()</code>	Seuillage	À partir d'une image en niveau de gris, le seuillage d'image peut être utilisé pour créer une image en noir et blanc (monochrome). On remplace chacun des pixels d'une image par un pixel noirs (0) ou blanc (255) en fonction d'un seuil fixé (par exemple 50).	 

4. Travail à effectuer

1. **Créer** un fichier python `filtre.py`.
2. **Écrire** une classe `filtre` à partir du diagramme de classe présenté page 2
NB : le corps des méthodes ne sera pas développé immédiatement. On utilisera l'instruction Python `pass` en attendant.
3. **Compléter** le constructeur de classe.
4. **Compléter** les méthodes suivantes et tester au fur et à mesure leur validité :
 - `size()` : retourne la taille en pixels d'une image sous forme de tuple largeur, hauteur
 - `width()` : retourne la largeur d'une image en pixels
 - `height()` : retourne la hauteur d'une image en pixels
 - `weight()` : retourne le poids d'une image en pixels
 - `get_pix(x, y)` : retourne la valeur du pixel de coordonnées (x,y), ou `None` si erreur
5. **Compléter** les méthodes de traitement d'image et les tester au fur et à mesure.